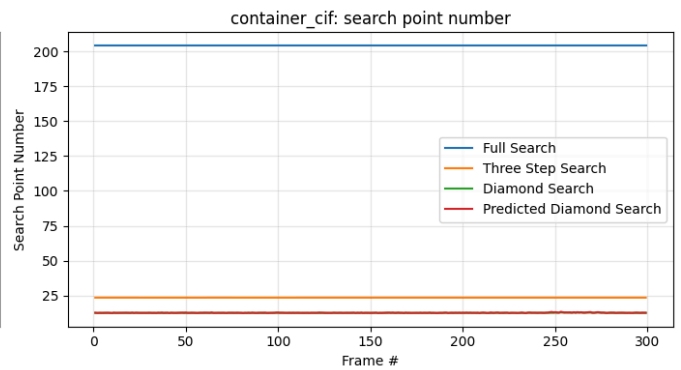
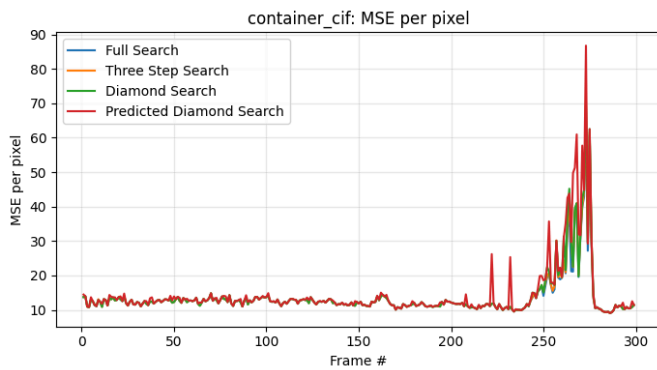
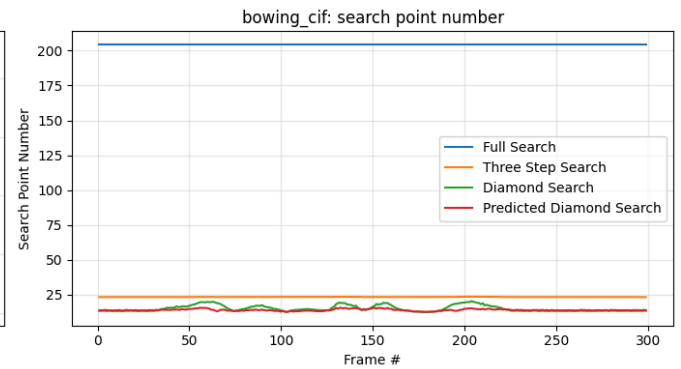
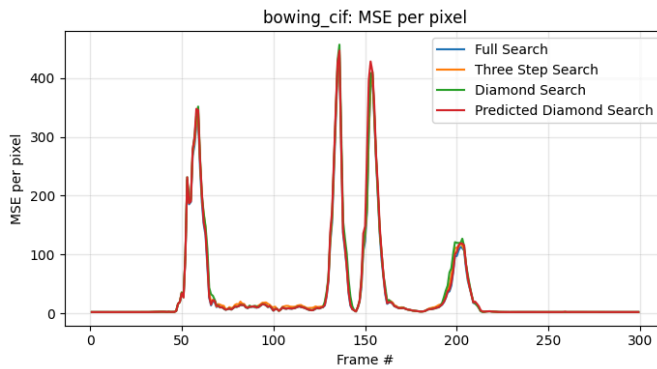
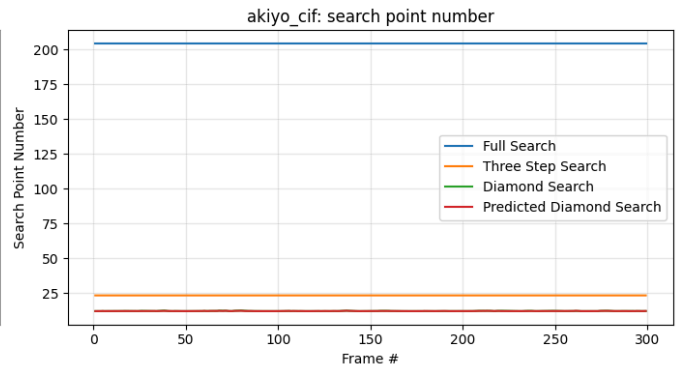
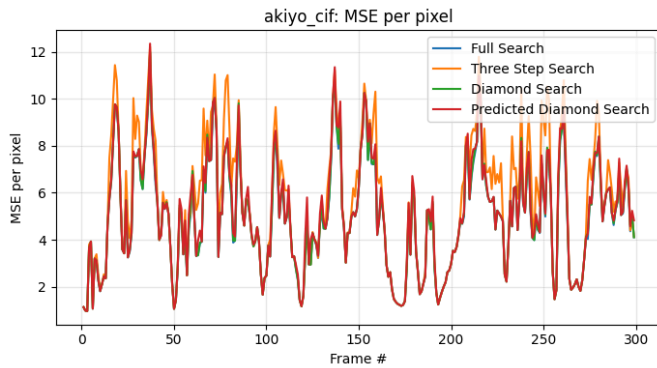


Hw5

12533581 苏子钰

本次作业尝试实现了四种block based的运动估计算法。以下是在search window = 15x15, 在akiyo_cif, bowing_cif与 container_cif下得到的结果。参考维基百科, 作业中实现的Diamond Search是分大小diamond的diamond search, 做出改动原本期望能削减MSE峰值, 但是实际并未达到期望的结果。



Average MSE per Pixel (Frame Distance = 1)

Algorithm	akiyo_cif	bowing_cif	container_cif
Full Search	5.007	34.274	13.463
Three Step Search	5.645	36.281	13.573
Diamond Search	5.045	37.331	13.682
Predicted Diamond Search	5.142	36.775	14.397

Average Search Point Number (Frame Distance = 1)

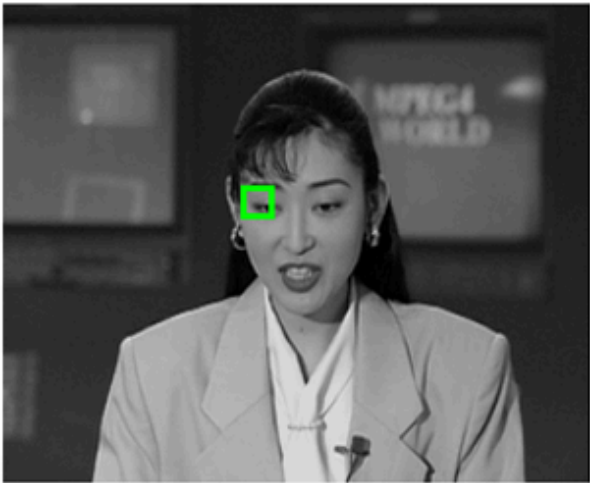
Algorithm	akiyo_cif	bowing_cif	container_cif
Full Search	204.28	204.28	204.28
Three Step Search	23.49	23.38	23.39
Diamond Search	12.28	15.09	12.41
Predicted Diamond Search	12.26	14.03	12.57

第一个表说明了几个算法都能取得接近的性能，这和理论预期是相符的，表二说明带有初始位置的Diamond Search在部分场景下会相较于不带初始位置的有一定的提升。

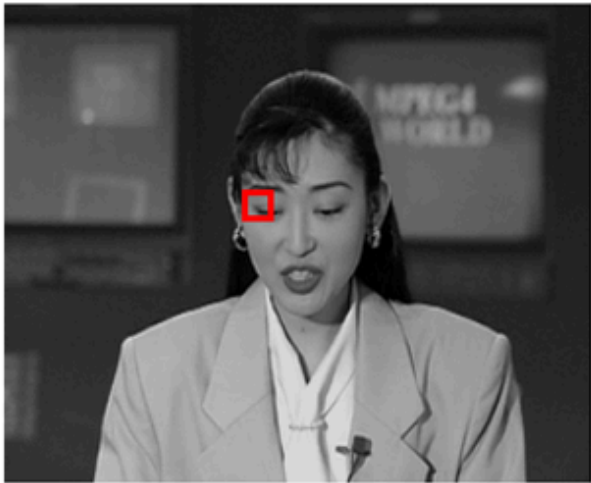
可以注意到在akiyo中，MSE都处于一个较为正常的值，而在bowing和container下，会出现MSE尖峰。最直观的想法是，记录算法每一步的输入和输出block，这样就能比较直观的了解到为什么会出现尖峰。

对于较为正常的akio，选取其MSE最大的两帧对比：

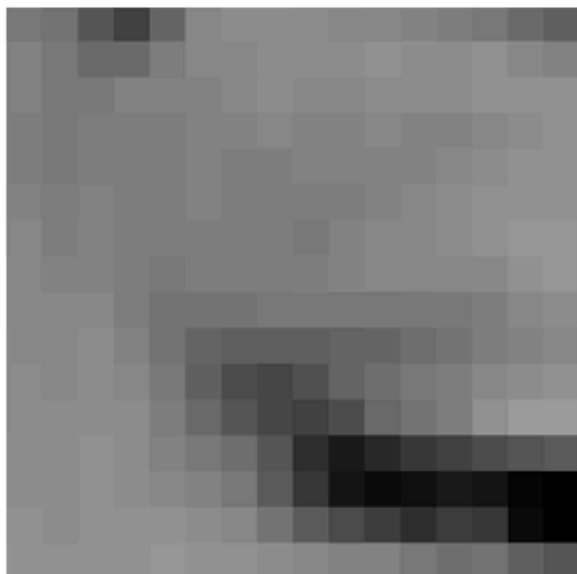
Previous Y frame



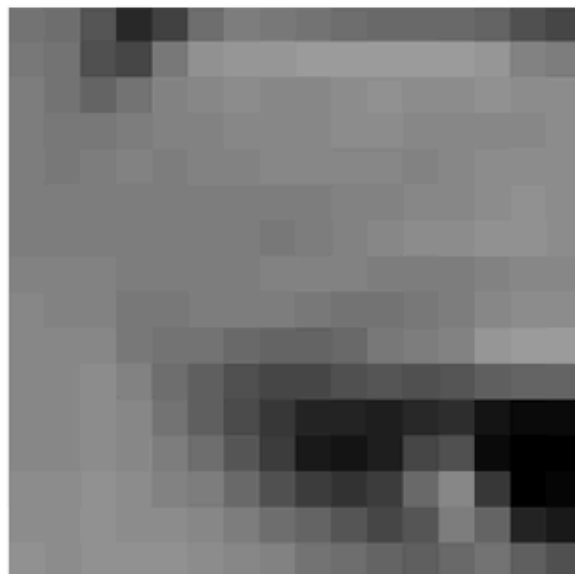
Current Y frame



Current block



Matched reference block

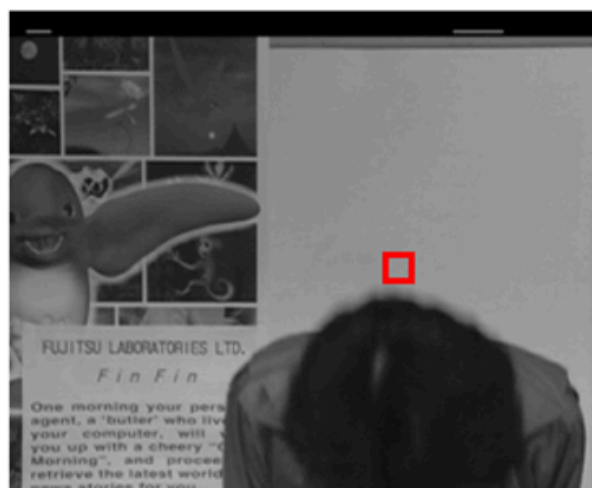


以上是放大的块图像，可以注意到两者的差距不大，这和MSE较低的观测相符合。而在bowing中，

Previous Y frame



Current Y frame



可以很明显的观测到表示运动估计的框并没有出现在应该出现的地方，由头发直接变成了背景

Current block



Matched reference block



这带来了很大的MSE，通过查看运动向量的值（7,-7）可知，算法触碰到了搜索边界仍然无法找到最优匹配，帧间运动超出了search window可以捕捉的范围。因此，可以通过增大search window来获得比较好的结果，遗憾的是，在增加到30之后，只获取了较原来少有提升的结果，而无法削减峰值。而再次增大search range可能会导致整体运行时间过长。

另外，作业采用median mv的方式提供搜索先验，median mv通过计算过的邻近块的mv取中位数以获得对当前块的初始估计。实验结果表明，在特定帧的搜索中，先验可以明显地降低所需的计算步数。